

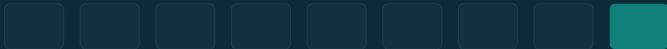


STREAM PROCESSING AVEC APACHE KAFKA

Kafka Streams

KStream, KTable et la topologie de traitement

ESGI · 5^e année · Data Engineering · 2025–2026



SÉANCE 9

Objectifs

- **Situer Kafka Streams**

Une bibliothèque Java qui s'exécute dans votre application — pas un cluster séparé.

- **Comprendre la topologie**

Un graphe source → traitements → sink, décrit avec le DSL.

- **Distinguer KStream et KTable**

Un flux de faits face à une table d'états, et la dualité entre les deux.

- **Écrire et lancer une première topologie**

Transformations *stateless* : `filter`, `mapValues`, `to`.

SÉANCE 9

Au programme

01

Bibliothèque & topologie

le DSL

02

KStream

transformations stateless

03

KTable & dualité

flux face à table

04

Atelier

filtrer, transformer

01

- PARTIE 1

La bibliothèque & la topologie

Une bibliothèque, pas un cluster

- Kafka Streams est un **jar** que vous ajoutez à votre application : elle lit des topics, transforme, et réécrit des topics.
- On **passe à l'échelle** en lançant plusieurs instances de l'application : même `application.id` $\Rightarrow$ un *consumer group*, les partitions se répartissent.

■ Consumer

Bas niveau : vous gérez tout (offsets, état, parallélisme).

■ Connect

Intégration *sans code* entre Kafka et l'extérieur.

Streams

Traitement *riche par code*, dans votre application.

Une topologie de traitement



- Une application Streams est un **graphe** : des *sources* (topics), des *processeurs* (les opérations), des *sinks* (topics).
- Le **DSL** (`StreamsBuilder`) décrit ce graphe de façon déclarative ; la *Processor API* offre un contrôle bas niveau.

02

- PARTIE 2

KStream — transformations stateless

Un flux de faits

- Un `KStream` est une suite **non bornée** d'enregistrements : chaque message est un **fait** indépendant (un événement qui s'ajoute).
- Les transformations **stateless** traitent chaque enregistrement *isolément*, sans mémoire d'un message à l'autre.

■ Par valeur

`mapValues`, `filter`,
`flatMapValues`, `peek`, `foreach`.

■ Sur la clé

`selectKey`, `map`, et le branchement avec
`split()`.

Décrire la topologie avec le DSL

Source -> filter -> mapValues -> sink

```
StreamsBuilder builder = new StreamsBuilder();

KStream<String, String> orders =
    builder.stream("orders",
        Consumed.with(Serdes.String(), Serdes.String()));

orders.filter((key, line) -> amountOf(line) >= 10.0)
    .mapValues(app::upperCurrency)
    .to("orders-filtered",
        Produced.with(Serdes.String(), Serdes.String()));
```

Chaque opérateur renvoie un nouveau `KStream` : on enchaîne, on ne mute pas.

Configurer et lancer l'application

Propriétés essentielles, puis démarrage

```
Properties p = new Properties();  
p.put("application.id",      "tp9-order-stream"); // = consumer group  
p.put("bootstrap.servers",   "localhost:29092");  
p.put("default.key.serde",   stringSerdeClass);  
p.put("default.value.serde", stringSerdeClass);  
  
KafkaStreams streams = new KafkaStreams(builder.build(), p);  
Runtime.getRuntime().addShutdownHook(new Thread(streams::close));  
streams.start();
```

`application.id` identifie le groupe et le répertoire d'état local. Garantie : `exactly_once_v2` en option.

Produire, lancer, consommer

Lancer la topologie, l'alimenter, lire la sortie

```
mvn -q compile exec:java \  
  -Dexec.mainClass=fr.esgi.kafka.tp9.OrderStreamApp &  
echo 'o-1;19.9;eur' | kafka-console-producer.sh \  
  --bootstrap-server localhost:29092 --topic orders  
kafka-console-consumer.sh --bootstrap-server localhost:29092 \  
  --topic orders-filtered --from-beginning
```

■ Filtré

Une commande < 10 n'apparaît pas dans `orders-filtered`.

■ Transformé

`eur` ressort en EUR : `mapValues` a fait son travail.

03

- PARTIE 3

KTable & dualité

Une table d'états

- Un `KTable` interprète un topic comme une **table** : pour chaque clé, on ne garde que la **dernière valeur** (sémantique *upsert*).
- C'est le pendant des topics **compactés** (séance 3) : le changelog devient un état courant interrogeable.

■ KStream — un fait

Clé `k` reçue deux fois = **deux** enregistrements (on ajoute).

KTable — un état

Clé `k` reçue deux fois = **une** valeur, la dernière (on met à jour).

Un flux et une table sont deux vues



- Un flux **agrégé** devient une table (un compteur, un cumul) ; une table relue **enregistrement par enregistrement** redevient un flux.
- Cette **dualité** est au cœur de Kafka Streams : on passe de l'un à l'autre selon la question posée.

04

- PARTIE 4

Atelier — votre première topologie

Cluster et topics

Demarrer le cluster et creer les topics du TP

```
docker compose up -d          # 3 brokers KRaft

kafka-topics.sh --bootstrap-server localhost:29092 \
  --create --topic orders --partitions 3 --replication-factor 3
kafka-topics.sh --bootstrap-server localhost:29092 \
  --create --topic orders-filtered --partitions 3 --replication-factor 3
```

■ Projet

Maven + kafka-streams (sur Maven Central, pas de dépôt Confluent).

■ Entrée / sortie

Lignes `orderId;amount;currency`
dans `orders`.

Compléter, lancer, observer

- 1 Compléter la **configuration** : `application.id`, `bootstrap.servers`, `serdes` par défaut.
- 2 Compléter la **topologie** : `filter` (montant ≥ 10) puis `mapValues` (devise en majuscules).
- 3 Écrire le résultat avec `to("orders-filtered")` et lancer l'application.
- 4 Produire des commandes et vérifier le contenu de `orders-filtered`.
- 5 Lancer une **deuxième instance** (même `application.id`) et observer la répartition.

Le parallélisme en action

Voir les groupes et la repartition des partitions

```
# Le consumer group porte le nom de l'application.id
kafka-consumer-groups.sh --bootstrap-server localhost:29092 --list

# Detail : membres, partitions assignees, retard (lag)
kafka-consumer-groups.sh --bootstrap-server localhost:29092 \
  --describe --group tp9-order-stream
```

■ Un groupe

L'application.id apparaît comme un consumer group classique.

■ Réparti

Les partitions se distribuent entre les instances actives.

Pour aller plus loin

Objectif — enrichir la topologie et toucher du doigt la repartition et l'Avro.

■ 1. Brancher

Avec `split()`, router les commandes vers deux topics selon la devise.

■ 2. Reclé

`selectKey` sur la devise, puis observer la repartition.

■ 3. Avro

Brancher un `SpecificAvroSerde` sur le registre de la séance 7.

Au-delà du stateless

- **Le traitement avec état** : compter, cumuler, **joindre** des flux, fenêtrer dans le temps — appuyé sur des *state stores* sauvegardés dans des changelogs.
- **Interactive Queries** : interroger directement l'état local de l'application, sans repasser par un topic.
- **Alternatives** : `ksqlDB` (SQL sur les flux) et Apache `Flink` pour d'autres compromis.

■ Stateless

Aujourd'hui : transformer chaque enregistrement isolément.

■ Stateful

À venir : agréger, joindre et fenêtrer des flux.

SÉANCE 9

À retenir

- ✓ Kafka Streams est une **bibliothèque** embarquée ; on scale en lançant plusieurs instances (même `application.id`).
- ✓ Une application = une **topologie** source → processeurs → sink, décrite par le DSL.
- ✓ `KStream` = un flux de **faits** (append) ; `KTable` = une table d'**états** (upsert).
- ✓ Dualité : on passe du flux à la table (**agrégation**) et inversement (**changelog**).
- ✓ Stateless aujourd'hui : `filter`, `mapValues`, `to` ; `mapValues` préserve la clé.



LA SUITE — SÉANCE 10

TP — flux stateless en pratique

Router, filtrer et enrichir plusieurs flux dans une même topologie, et passer à l'échelle.